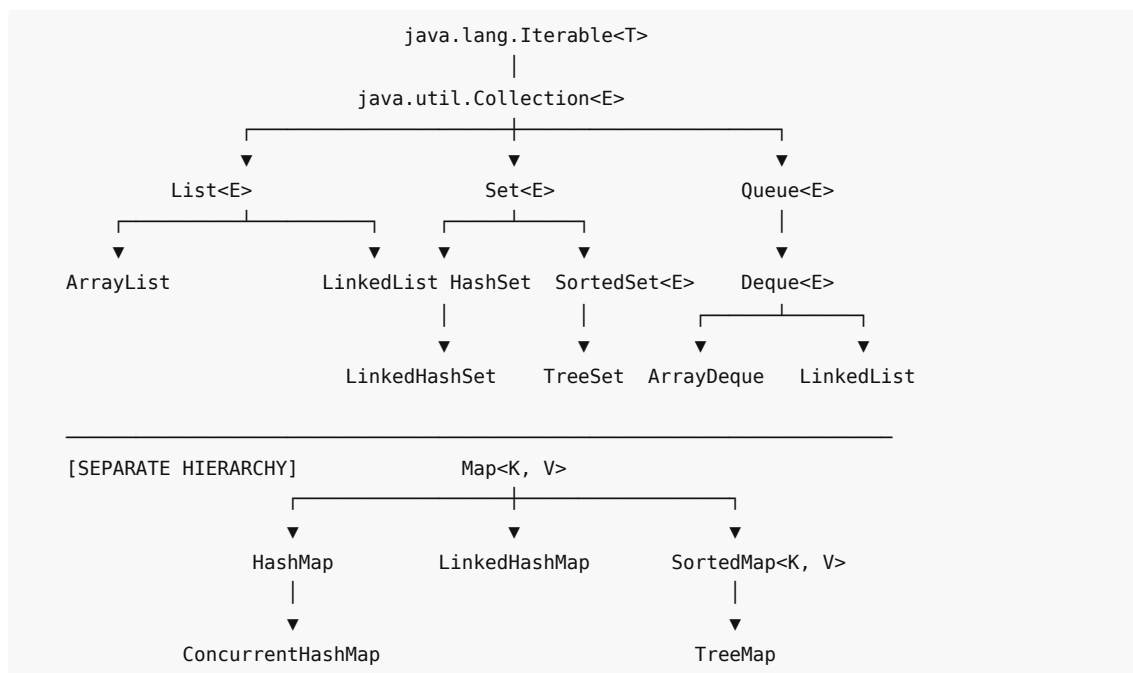


SECTION 4 — JAVA COLLECTIONS FRAMEWORK (Senior SDET Masterclass)

Topics Covered

- 4.1 Collection Framework Hierarchy & Architecture
- 4.2 Core Contracts: List, Set, Map, Queue, and Deque
- 4.3 ArrayList (Dynamic Resizing, Memory Compaction, & modCount Fail-Fast)
- 4.4 LinkedList (Node Overhead, Cache Locality Traps, & ArrayDeque Contrast)
- 4.5 CopyOnWriteArrayList (Thread-Safe Listeners & Snapshot Iterators)
- 4.6 HashSet (Backing HashMap Mechanics & Dummy Value Idioms)
- 4.7 LinkedHashMap (Preserving Insertion Order in Test Suites)
- 4.8 TreeSet (Red-Black Tree, Comparable, & Comparator Constraints)
- 4.9 HashMap Overview (Core Architecture & High-Concurrency Traps)
- 4.10 LinkedHashMap (LRU Test Execution Cache Implementation)
- 4.11 TreeMap (Navigable Map & Range-Based Test Partitioning)
- 4.12 ConcurrentHashMap (Java 8 CAS + Bucket-Level Synchronization)
- 4.13 Queue Implementations (Producer-Consumer Test Data Dispatch)
- 4.14 Deque & ArrayDeque (Ring Buffers vs. Legacy Stack)
- 4.15 PriorityQueue (Binary Heap for Risk-Based Test Scheduling)
- 4.16 Structural Comparisons (ArrayList vs. LinkedList)
- 4.17 Set Comparisons (HashSet vs. LinkedHashSet vs. TreeSet)
- 4.18 Map Comparisons (HashMap vs. LinkedHashMap vs. TreeMap)
- 4.19 Concurrency Comparisons (HashMap vs. Hashtable vs. ConcurrentHashMap)
- 4.20 Senior Architectural Collection Selection Matrix
- 4.21 Time & Space Complexity Master Reference (Big-O)
- 4.22 Collections in Test Automation Frameworks (Parallel Triage Playbook)

4.1 Collection Framework Hierarchy & Architecture



Key Architectural Distinctions Every Senior SDET Must Know

- 1. **Map is NOT a Collection** : `Map<K, V>` does not extend `Collection<E>` . Maps represent key-value pairings (two dimensions), whereas `Collection` represents singular elements. Maps project collection views via `keySet()` , `values()` , and `entrySet()` .
- 2. **Iterable<T> Contract**: Root of the collection hierarchy. Requires implementing `Iterator<T>` `iterator()` , enabling enhanced `for-each` loops.
- 3. **Collections (Utility) vs. Collection (Interface)**: `java.util.Collections` is an un-instantiable utility class containing static helper algorithms (`sort` , `synchronizedMap` , `unmodifiableList`).

4.2 Core Contracts: List, Set, Map, Queue, and Deque

Interface	Insertion Order	Duplicates Allowed	Primary Access Pattern	Typical SDET Use Case
List<E>	Preserved (Indexed).	Yes.	Positional index (<code>get(i)</code>).	Test execution steps, API JSON payload lists.
Set<E>	Unordered (except Linked/Tree).	No (Unique).	Membership check (<code>contains(x)</code>).	Unique user IDs, deduplicated error logs.
Map<K, V>	Unordered (except Linked/Tree).	Unique Keys; Duplicate Values.	Key lookup (<code>get(key)</code>).	Test context, environment configs, locators.
Queue<E>	FIFO (First-In, First-Out).	Yes.	Head retrieval (<code>poll()</code> , <code>peek()</code>).	Parallel test task dispatch, retry queues.
Deque<E>	Double-ended (FIFO / LIFO).	Yes.	Head/Tail (<code>pollFirst()</code> , <code>pollLast()</code>).	Browser navigation history, call-stack tracking.

4.3 ArrayList

1. Theory & Resizing Mechanics

`ArrayList<E>` is backed by a contiguous `Object[] elementData` array.

- **Initial Capacity**: Default is 10 (allocated lazily upon the first `.add()` invocation).
- **Growth Formula**: In Java 8+, growth computes as:
$$\text{newCapacity} = \text{oldCapacity} + (\text{oldCapacity} \gg 1) \approx 1.5 \times \text{growth}$$
- When capacity is exceeded, a new array is allocated and elements are copied using the native CPU-optimized call `System.arraycopy()` .
- **Fail-Fast Mechanics**: Maintains an internal counter `protected transient int modCount` . If `modCount` changes while an `Iterator` is traversing (structural modification), the iterator immediately throws `ConcurrentModificationException` .

```
// Sizing ArrayList defensively in enterprise test parsers
// AVOID: ArrayList resizing multiple times when loading 10,000 Excel rows
List<TestRow> rows = new ArrayList<>(10_000); // Pre-allocate initial capacity
```

4.4 LinkedList

1. Theory & The Memory / Cache Locality Trap

`LinkedList<E>` is implemented as a doubly-linked list.

- Each element is wrapped inside a `Node<E>` containing three 64-bit pointers: `item` , `next` , and `prev` .

- **Memory Inflation:** In a 64-bit JVM, each `Node` object consumes **24 bytes of memory overhead** plus the 8-byte pointer to the object. An `ArrayList` of 100,000 elements consumes $\approx 400\text{KB}$; a `LinkedList` consumes $\approx 3.2\text{MB}$!
- **CPU Cache Misses:** Elements in an `ArrayList` are contiguous in physical RAM, maximizing CPU L1/L2 prefetching. Nodes in a `LinkedList` are scattered randomly across the heap, causing CPU cache misses on every traversal.

[!TIP] **Senior Architecture Rule:** Almost never use `LinkedList` in modern Java. If you need a Queue or Stack, use `ArrayDeque`. If you need a List, use `ArrayList`.

4.5 CopyOnWriteArrayList

1. Theory & Concurrency Mechanics

`CopyOnWriteArrayList<E>` is a thread-safe variant of `ArrayList` located in `java.util.concurrent`.

- **Write Operation:** Every mutating operation (`add()` , `set()` , `remove()`) creates a brand-new underlying array copy using an internal `ReentrantLock`.
- **Read Operation:** Read operations (`get()` , `iterator()`) are completely lock-free and operate on a snapshot array at the moment the iterator was created.
- **Fail-Safe Iterators:** Iterators **never throw `ConcurrentModificationException`** and do not reflect subsequent writes. Mutating methods on the iterator (`it.remove()`) throw `UnsupportedOperationException`.

2. Production Code: Thread-Safe Test Lifecycle Event Broadcaster

```
package com.deloitte.sdet.listeners;

import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public final class TestEventDispatcher {

    // Thread-safe listener registration list. Read-heavy, write-rare.
    private final List<TestLifecycleListener> listeners = new CopyOnWriteArrayList<>();

    public void registerListener(TestLifecycleListener listener) {
        listeners.add(listener);
    }

    public void broadcastTestPassed(String testId) {
        // Safe lock-free iteration across 50 parallel execution threads
        for (TestLifecycleListener listener : listeners) {
            listener.onPass(testId);
        }
    }

    public interface TestLifecycleListener { void onPass(String testId); }
}
```

4.6 HashSet , 4.7 LinkedHashSet , and 4.8 TreeSet

Set Type	Backing Structure	Ordering	`contains()`	Special Requirements

`HashSet`	`HashMap`	None	$O(1)$ avg	`equals()` & `hashCode()`
`LinkedHashSet`	`LinkedHashMap`	Insertion	$O(1)$ avg	`equals()` & `hashCode()`
`TreeSet`	`TreeMap` (RB)	Sorted Order	$O(\log N)$	`Comparable`/`Comparator`

The HashSet Internal Secret

`HashSet` does NOT implement hashing from scratch. It delegates completely to a private `HashMap` :

```
// OpenJDK HashSet implementation detail
private transient HashMap<E, Object> map;
private static final Object PRESENT = new Object(); // Dummy constant

public boolean add(E e) {
    return map.put(e, PRESENT) == null;
}
```

4.10 LinkedHashMap (LRU Test Execution Cache)

`LinkedHashMap` maintains a doubly-linked list running through all of its entries. By overriding `removeEldestEntry()` , you can construct an automated **Least-Recently-Used (LRU) Cache** in 10 lines of code.

Production Code: LRU Browser Session / Token Cache

```
package com.deloitte.sdet.cache;

import java.util.LinkedHashMap;
import java.util.Map;

public class LruTestTokenCache extends LinkedHashMap<String, String> {

    private final int maxEntries;

    public LruTestTokenCache(int maxEntries) {
        // accessOrder = true: ordered by access frequency, not insertion order
        super(maxEntries, 0.75f, true);
        this.maxEntries = maxEntries;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<String, String> eldest) {
        // Automatically evicts the least recently accessed token when limit exceeded
        return size() > maxEntries;
    }
}
```

4.12 ConcurrentHashMap

In multi-threaded test frameworks running across 16-32 parallel workers:

- **HashMap** : Non-thread-safe. Multiple writers corrupt table pointers or cause lost updates.
- **Hashtable** / **Collections.synchronizedMap()** : Coarse-grained locking. A single monitor lock blocks ALL reading and writing threads, serializing parallel test execution.
- **ConcurrentHashMap (Java 8+)**:
 - **Reads**: 100% lock-free via `volatile` reads.
 - **Writes to Empty Buckets**: Lock-free using CPU hardware **CAS (Compare-And-Swap)**.
 - **Writes to Populated Buckets**: Locks **ONLY** the **head node of that specific bucket** using a fine-grained `synchronized` block. Multiple threads can write concurrently to different buckets.

4.13 Queue & 4.14 ArrayDeque

Method Function	Throws Exception	Returns Special Value (`false` / `null`)
Insert at Tail	<code>`add(e)`</code>	<code>`offer(e)`</code>
Remove from Head	<code>`remove()`</code>	<code>`poll()`</code>
Examine Head	<code>`element()`</code>	<code>`peek()`</code>

[!WARNING] Always use `offer()` and `poll()` in test framework queues. `add()` throws an unchecked `IllegalStateException` when a bounded queue is full; `remove()` throws `NoSuchElementException` when empty.

4.15 PriorityQueue (Risk-Based Test Scheduling)

`PriorityQueue<E>` is an unbounded priority heap backed by a dynamic array representing a **binary min-heap**:

- Root element is always the minimum element according to natural ordering or custom `Comparator` .
- Time complexity: $O(\log N)$ for `offer()` and `poll()` ; $O(1)$ for `peek()` .

Production Code: Executing Critical Flaky Tests First

```
package com.deloitte.sdet.scheduler;

import java.util.PriorityQueue;

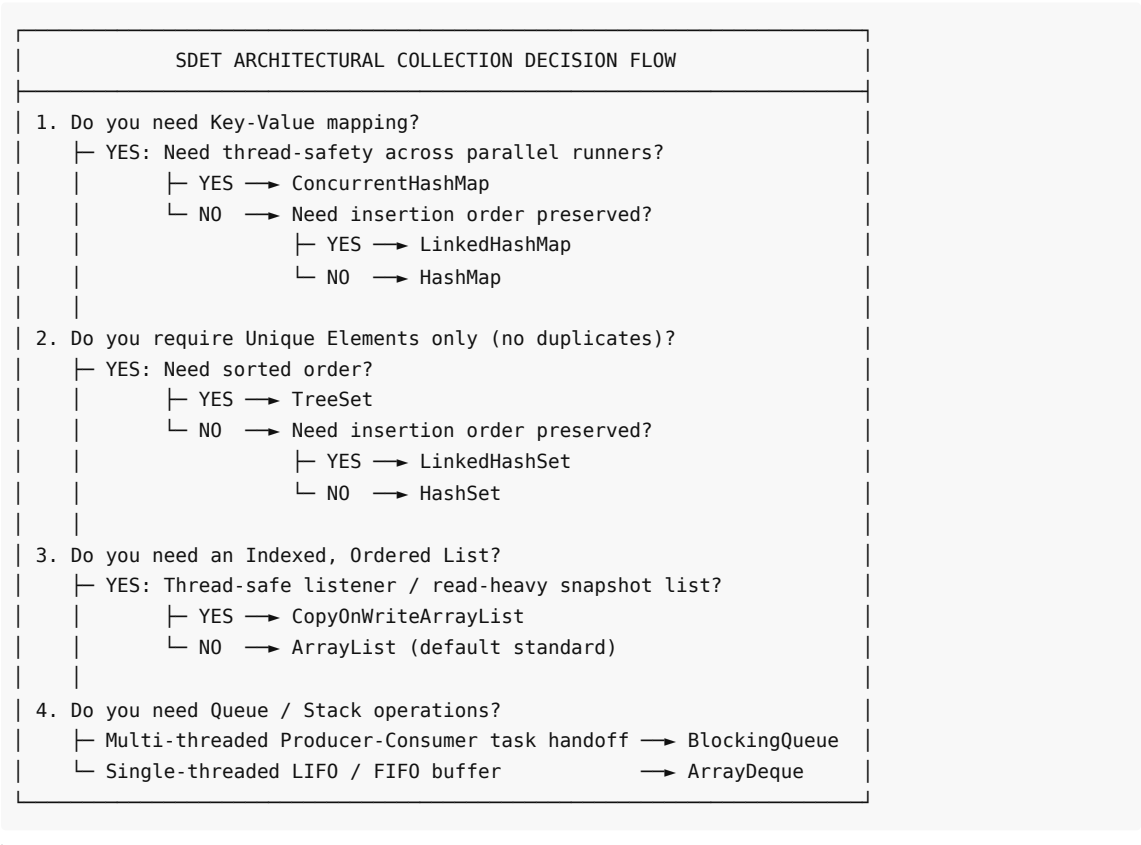
public record TestCase(String testId, int riskPriority) implements Comparable<TestCase> {
    @Override
    public int compareTo(TestCase other) {
        // Priority 1 executes before Priority 2
        return Integer.compare(this.riskPriority, other.riskPriority);
    }
}

public class TestScheduler {
    public static void main(String[] args) {
        PriorityQueue<TestCase> queue = new PriorityQueue<>();
        queue.offer(new TestCase("TC-300", 3)); // Low priority
        queue.offer(new TestCase("TC-101", 1)); // Blocker/P1
        queue.offer(new TestCase("TC-202", 2)); // Major

        while (!queue.isEmpty()) {
            System.out.println("Executing: " + queue.poll().testId());
        }
        // Output: TC-101, then TC-202, then TC-300
    }
}
```

```
}  
}
```

4.20 Senior Architectural Collection Selection Matrix



4.21 Time & Space Complexity Master Reference (Big-O)

Collection	get() / search	add() / insert	remove()	Space Complexity
ArrayList	$O(1)$ by index; $O(N)$ by value	$O(1)$ amortized; $O(N)$ worst	$O(N)$ (requires array shift)	$O(N)$ contiguous
LinkedList	$O(N)$	$O(1)$ at head/tail; $O(N)$ middle	$O(1)$ at head/tail; $O(N)$ middle	$O(N)$ with 24B/node
ArrayDeque	$O(1)$ at head/tail	$O(1)$ amortized	$O(1)$ at head/tail	$O(N)$ ring buffer
HashSet	$O(1)$ average; $O(N)$ worst	$O(1)$ average; $O(N)$ worst	$O(1)$ average; $O(N)$ worst	$O(N)$
TreeSet	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$ Red-Black tree
HashMap	$O(1)$ average; $O(\log N)$ modern	$O(1)$ average; $O(\log N)$ modern	$O(1)$ average; $O(\log N)$ modern	$O(N)$

TreeMap	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$ Red-Black tree
ConcurrentHashMap	$O(1)$ average	$O(1)$ average	$O(1)$ average	$O(N)$

4.22 High-Stakes Concurrency Scenarios in Test Frameworks

Scenario 1: ConcurrentModificationException during Parallel Reporting

- Symptom:** Test execution passes, but the TestNG/ExtentReport listener fails with `java.util.ConcurrentModificationException` during `@AfterSuite`.
- Root Cause:** The custom reporter uses a standard `ArrayList<TestResult>`. As 16 worker threads finish tests and call `list.add()`, a background reporting thread traverses the list via `for (TestResult r : list)`, detecting a mismatched `modCount`.
- Senior Resolution:** Replace `ArrayList` with `ConcurrentLinkedQueue<TestResult>` or `CopyOnWriteArrayList<TestResult>`.

Scenario 2: Deadlock with Collections.synchronizedMap()

- Symptom:** Test runner hangs indefinitely on 8 parallel threads during token refresh.
- Root Cause:** Two different threads locked the same synchronized map while executing nested operations requiring another monitor lock in reverse order.
- Senior Resolution:** Replace `Collections.synchronizedMap` with `ConcurrentHashMap` and utilize atomic compute methods (`computeIfAbsent`, `merge`) instead of compound `if (!map.containsKey(k)) map.put(k, v)` blocks.

4.10 Exhaustive java.util.Arrays Method Master Reference

Method Signature	Return Type	Description & SDET Use Case	Time Complexity
<code>Arrays.sort(T[] a)</code>	<code>void</code>	Dual-Pivot Quicksort for primitives, Timsort for objects.	$O(N \log N)$
<code>Arrays.binarySearch(T[] a, T key)</code>	<code>int</code>	Binary search on sorted array; returns index or $-(\text{insertionPoint}) - 1$.	$O(\log N)$
<code>Arrays.asList(T... a)</code>	<code>List<T></code>	Returns fixed-size list backed by specified array (mutation throws <code>UnsupportedOperationException</code>).	$O(1)$
<code>Arrays.copyOf(T[] original, int newLength)</code>	<code>T[]</code>	Copies array, truncating or padding with nulls/zeros.	$O(N)$
<code>Arrays.copyOfRange(T[] a, int from, int to)</code>	<code>T[]</code>	Copies specified range of array into a new array.	$O(K)$
<code>Arrays.equals(T[] a, T[] a2)</code>	<code>boolean</code>	Structural equality check of array elements.	$O(N)$
<code>Arrays.deepEquals(Object[] a1, Object[] a2)</code>	<code>boolean</code>	Deep structural equality check for multi-dimensional arrays.	$O(N)$
<code>Arrays.fill(T[] a, T val)</code>	<code>void</code>	Assigns specified value to each element of array.	$O(N)$
<code>Arrays.stream(T[] array)</code>	<code>Stream<T></code>	Converts array into sequential Java 8 Stream.	$O(1)$

<code>Arrays.mismatch(T[] a, T[] b)</code> (Java 9+)	<code>int</code>	Finds index of first mismatch between two arrays, or -1.	$O(N)$
<code>System.arraycopy(src, sPos, dest, dPos, len)</code>	<code>void</code>	Native memory copy from source array to destination array.	$O(N)$ native

4.11 Exhaustive `java.util.Collections` Method Master Reference

Method Signature	Return Type	Description & SDET Use Case	Time Complexity
<code>Collections.sort(List<T> list)</code>	<code>void</code>	Sorts list in ascending order using Timsort.	$O(N \log N)$
<code>Collections.binarySearch(List list, T key)</code>	<code>int</code>	Binary search on sorted List ($O(\log N)$ for RandomAccess, $O(N)$ for LinkedList).	$O(\log N)$ / $O(N)$
<code>Collections.reverse(List<?> list)</code>	<code>void</code>	Reverses order of elements in list.	$O(N)$
<code>Collections.shuffle(List<?> list)</code>	<code>void</code>	Randomly permutes elements in list (useful for test data randomization).	$O(N)$
<code>Collections.swap(List<?> list, int i, int j)</code>	<code>void</code>	Swaps elements at specified positions.	$O(1)$
<code>Collections.max(Collection<? extends T> coll)</code>	<code>T</code>	Returns maximum element according to natural order.	$O(N)$
<code>Collections.min(Collection<? extends T> coll)</code>	<code>T</code>	Returns minimum element according to natural order.	$O(N)$
<code>Collections.unmodifiableList(List<? extends T> list)</code>	<code>List<T></code>	Returns unmodifiable view of specified list.	$O(1)$
<code>Collections.synchronizedList(List<T> list)</code>	<code>List<T></code>	Returns synchronized (thread-safe) list wrapper.	$O(1)$
<code>Collections.frequency(Collection<?> c, Object o)</code>	<code>int</code>	Returns number of elements in collection equal to o.	$O(N)$
<code>Collections.disjoint(Collection<?> c1, Collection<?> c2)</code>	<code>boolean</code>	Returns true if two collections have no elements in common.	$O(N)$
<code>Collections.emptyList()</code>	<code>List<T></code>	Returns immutable empty list (prevents null return anti-pattern).	$O(1)$

4.12 Exhaustive `java.util.Objects` Method Master Reference

Method Signature	Return Type	Description & SDET Use Case	Time Complexity
<code>Objects.equals(Object a, Object b)</code>	<code>boolean</code>	Null-safe equality check (<code>a == b (a != null && a.equals(b))</code>).	$O(1)$ / $O(N)$

<code>Objects.deepEquals(Object a, Object b)</code>	boolean	Null-safe deep structural equality check for nested objects/arrays.	$O(N)$
<code>Objects.hashCode(Object o)</code>	int	Null-safe hash code computation (<code>o == null ? 0 : o.hashCode()</code>).	$O(1)$
<code>Objects.hash(Object... values)</code>	int	Generates composite hash code for multiple fields.	$O(N)$
<code>Objects.requireNonNull(T obj, String msg)</code>	T	Checks non-nullity; throws <code>NullPointerException</code> with custom message.	$O(1)$
<code>Objects.requireNonNullElse(T obj, T defaultObj)</code>	T	Returns <code>obj</code> if non-null, else returns <code>defaultObj</code> (Java 9+).	$O(1)$
<code>Objects.isNull(Object obj)</code>	boolean	Returns <code>true</code> if reference is <code>null</code> .	$O(1)$
<code>Objects.nonNull(Object obj)</code>	boolean	Returns <code>true</code> if reference is non-null.	$O(1)$